Course : AAD – ALGORITHM ANALYSIS AND DESIGN

Chapter 7

Design and analysis of Greedy algorithms

Dr Rahaf Ghazal

2025-2026

Introduction

- Greedy algorithm is a common approach to solving optimization problems.
- At every step of the algorithm, we make a choice that looks the best at the moment.
- To greedily make **locally optimal** decisions in the hope of obtaining a **globally optimal** solution.

When to Use Greedy algorithms?

Greedy algorithm is used for solving problems that consists of the following characteristics:

1. Optimal Substructure:

means that we use the optimal results of subproblems to achieve the optimal result of the bigger problem.

2. Greedy Choice Property:

The optimal solution can be constructed by making the best local choice at each step.

Note: Greedy algorithms do not always give the best solution. For example, in coin change and 0/1 knapsack problems, we get the best solution using Dynamic Programming.

Greedy vs. Dynamic algorithms

Greedy algorithms and dynamic programming are both commonly used to solve optimization problems. They share some similarities, such as both relying on the optimal substructure property, but they work differently:

- Dynamic programming considers all previous decisions when making the current decision, and uses solutions to past subproblems to construct the solution to the current subproblem.
- Greedy algorithms do not consider past decisions, but instead make greedy choices moving forward, continually reducing the problem size until the problem is solved.

We will first understand how greedy algorithms work through an example:

problem "coin change"

Let's say you have a set of coins with values **[1, 2, 5, 10]** and you need to give minimum number of coin to someone change for **39**.

Pseudocode:

```
Algorithm: MinCoins(coins, amount)
   n = size of coins
   Sort coins in descending order
   res = 0
For i from 0 to n - 1:
    {  If amount >= coins[i]:
          {   cnt = amount / coins[i]          // حساب كم قطعة نحتاج من هذه الفئة
              res = res + cnt                  // إضافة العدد للنتيجة وتحديث المبلغ المتبقي
             amount = amount - (cnt * coins[i])  }
      If amount == 0:                         // التوقف إذا وصلنا للمبلغ المطلوب
         break
    }
Return res
```

01 Step

We need to make change for **39** using coins of denominations **{1, 2, 5, 10}** using the fewest number of coins.



Remaining amount = 39

Greedy Algorithm

However !!

Examples where greedy algorithms cannot find the optimal solution

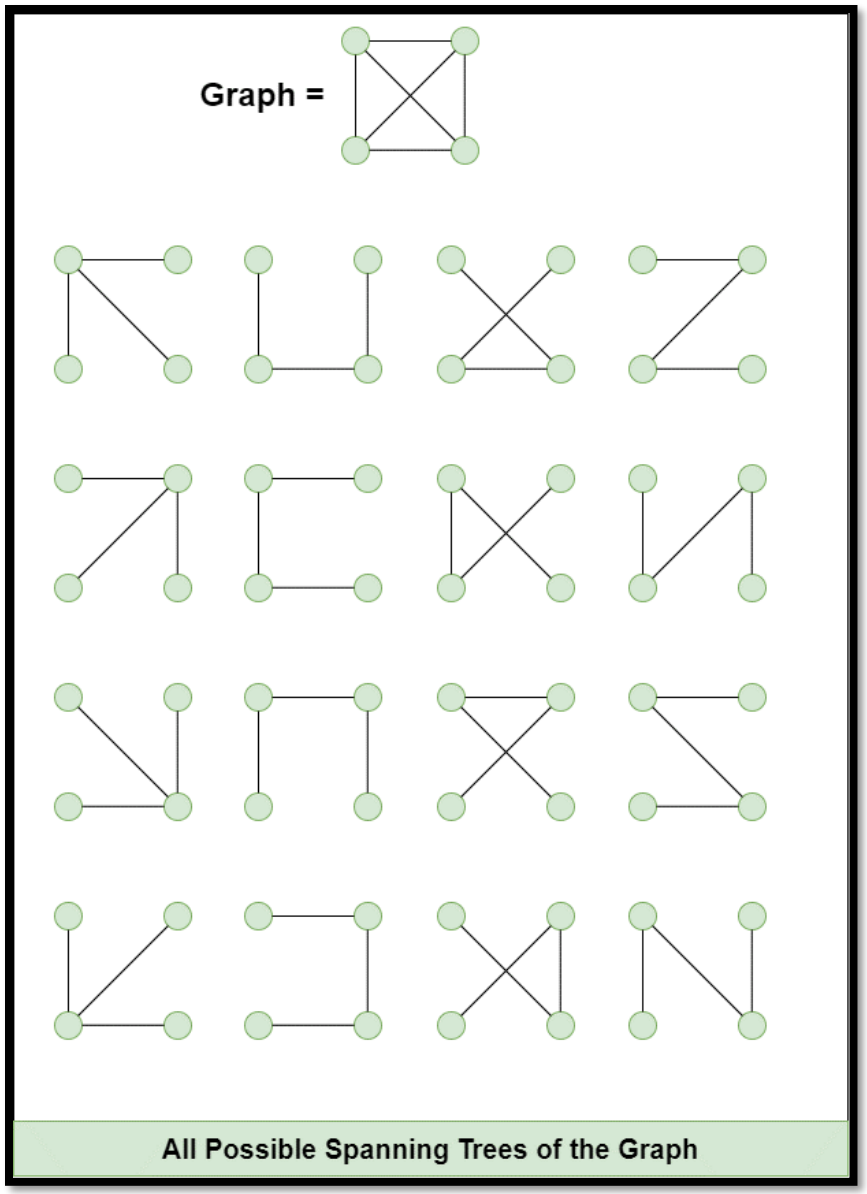
Coin combination	Target amount	Optimal greedy solution (Local optimum)	Optimal DP solution (Global optimum)
1, 20, 50	60	50 + 1 × 10	20 × 3
1, 49, 50	98	50 + 1 × 48	49 × 2

In other words, for the coin change problem, greedy algorithms cannot guarantee finding the global optimal solution, and may even find very poor solutions. It is better suited for solving with dynamic programming.

Common Algorithms that Use Greedy algorithms

- Minimum Spanning Tree (MST)Problem
  - Kruskal's algorithm
  - Prim's algorithm
- Dijkstra's algorithm

Minimum Spanning Tree (MST)Problem



What is a Spanning Tree?

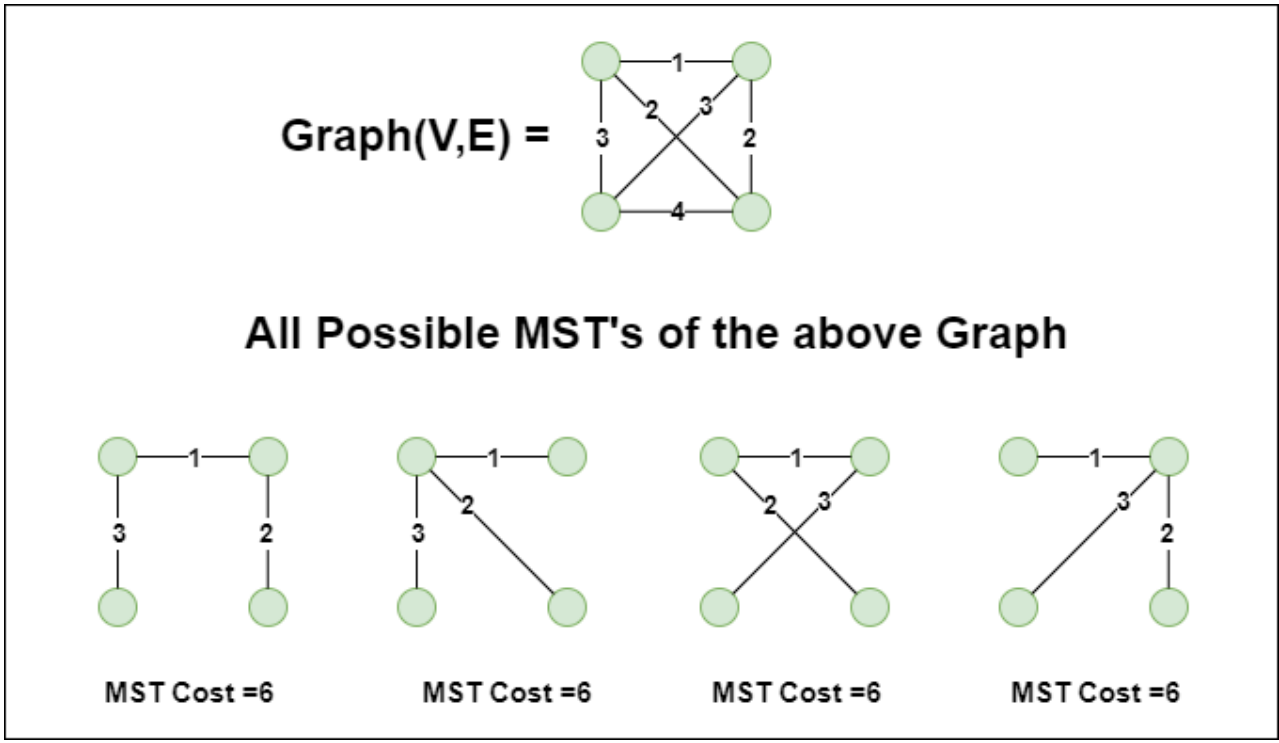
A spanning tree is a subset of Graph G, such that all the vertices are connected using minimum possible number of edges. Hence, a spanning tree does not have cycles and a graph may have more than one spanning tree.

Properties of a Spanning Tree:

- A spanning tree only exists for connected graphs.
- It must have **N-1** edges (where **N** is the number of vertices) and no cycles.
- The total number of possible spanning trees in a complete graph is N^{N-2}

Minimum Spanning Tree (MST)

- A minimum spanning tree (MST) is defined as a spanning tree that has the minimum weight among all the possible spanning trees.
- Like a spanning tree, there can also be many possible MSTs for a graph.



Key Application of MST

Designing cost-effective network infrastructure:

telephone, electrical, hydraulic, TV cable, computer, road

Common Algorithms

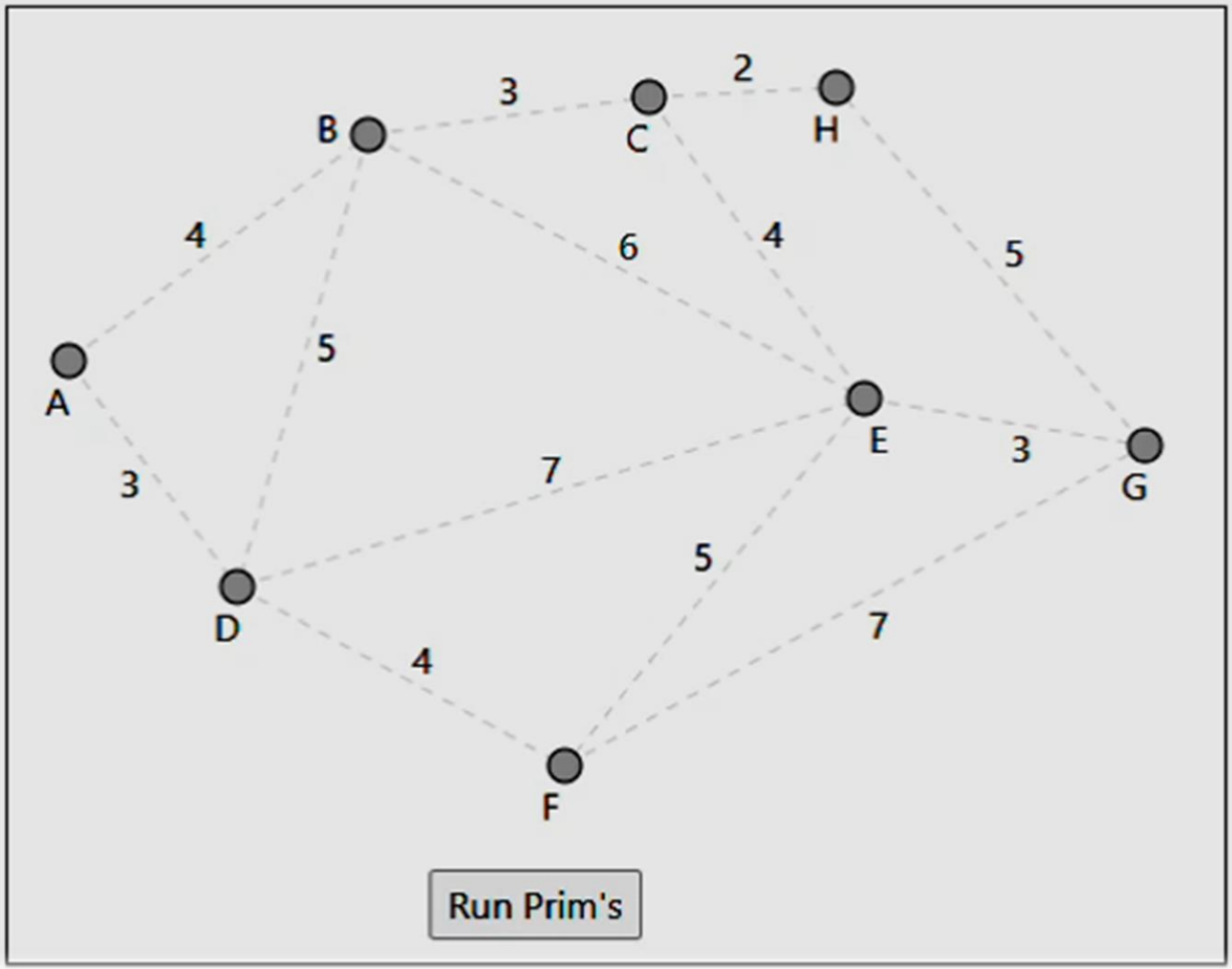
Two widely used greedy algorithms for finding the MST are:

Prim's Algorithm

Kruskal's Algorithm

Prim's Algorithm :

- Prim’s algorithm is a Greedy algorithm.
- Is to build the MST incrementally by starting with a single vertex and gradually extending the tree by adding the closest neighboring vertex at each step.

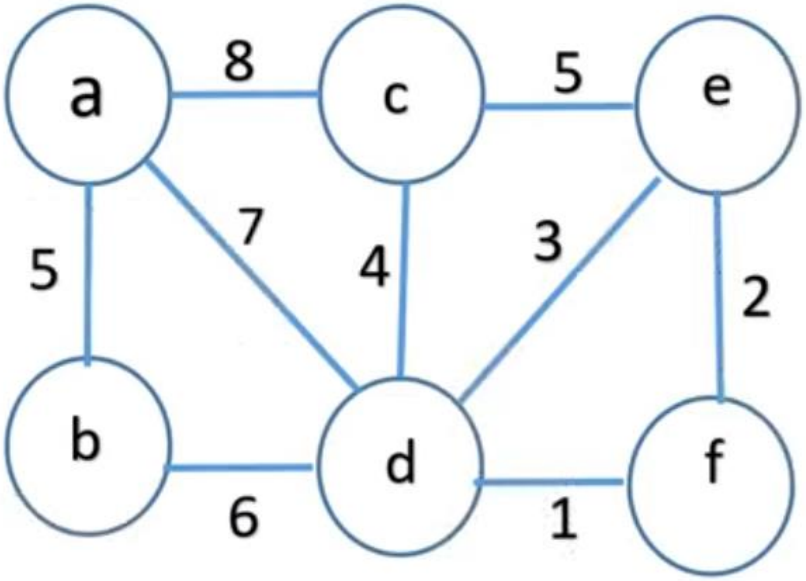


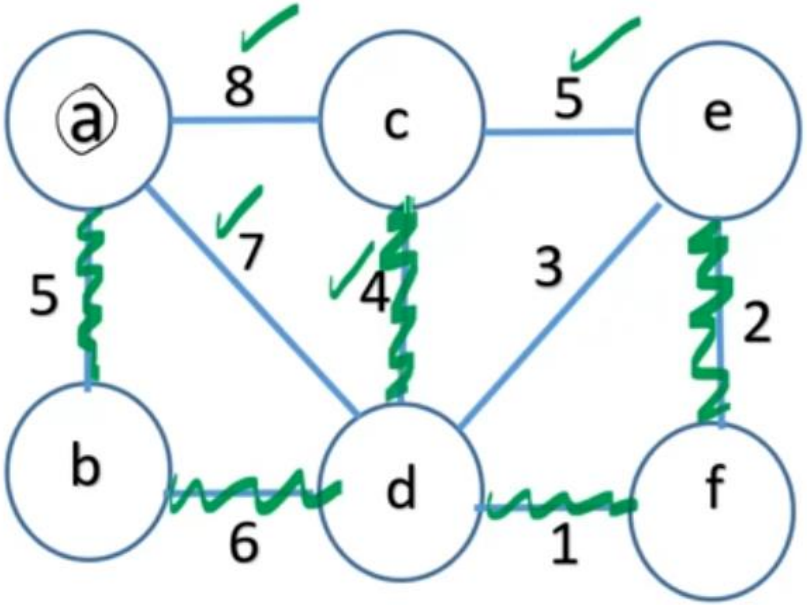
Pseudocode:

Algorithm Prim(Graph G, starting_vertex r):

1. For each vertex v in G:
   v.key = ∞
   v.parent = NULL
   Set v.visited = False // This array tracks visited vertices
2. r.key = 0
3. Initialize a min-priority queue Q containing all vertices in G,
   using their 'key' values as the priority.
4. While Q is not empty:
   u = Q.extractMin() // Get the vertex with the smallest key
   Set u.visited = True // Mark vertex u as "visited" and added to the MST
   For each neighbor v of u:
      If v.visited == False AND weight(u, v) < v.key:
         v.parent = u
         v.key = weight(u, v)
         Q.decreaseKey(v, v.key) // Update priority in the queue
5. Return the set of edges {(v.parent, v)} for all v except the root r.

Prim's Algorithm- Example

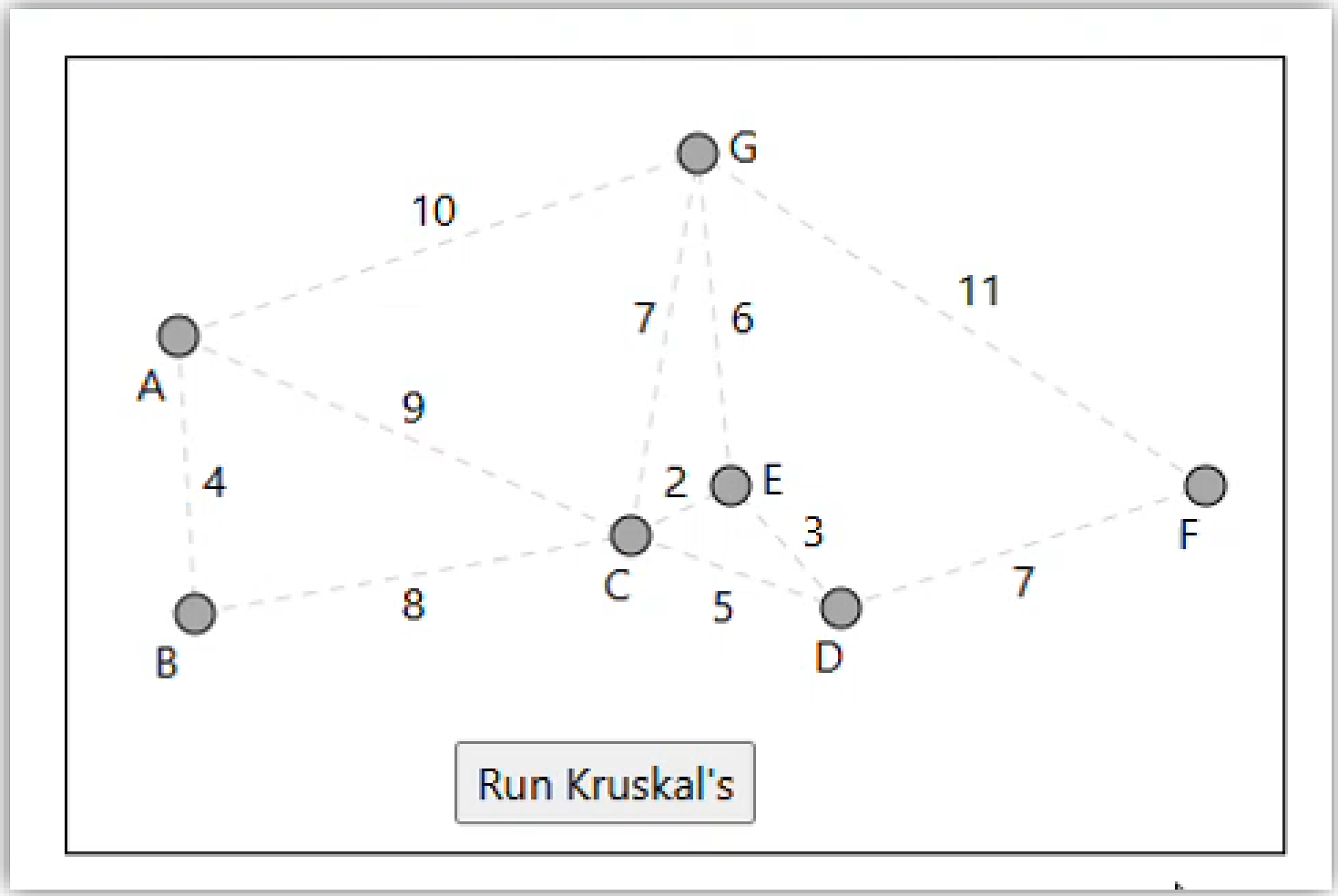




After processing, The final result will be:

Kruskal's Algorithm :

- Kruskal's Algorithm is a Greedy Algorithm.
- In each iteration It picks the minimum weighted edge at first and adds it to the growing spanning tree .



Pseudocode:

KRUSKAL(Graph G):

```
A = ∅                          // Initialize empty set for MST edges
for each vertex v in G.V:      // For each vertex, create a separate set
    MAKE-SET(v)

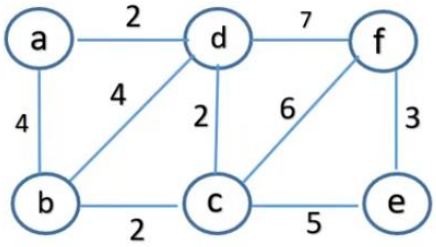
// Sort all edges in non-decreasing order of weight
sort the edges of G.E by weight w

for each edge (u, v) in G.E (in sorted order):
    // If u and v are in different sets, adding this edge won't form a cycle
    if FIND-SET(u) ≠ FIND-SET(v):
        A = A ∪ {(u, v)}       // Add edge to MST
        UNION(u, v)            // Merge the two sets into one component

return A                       // Return the set of edges in the MST
```

Kruskal's Algorithm- Example:

V = {a, b, c, d, e, f }

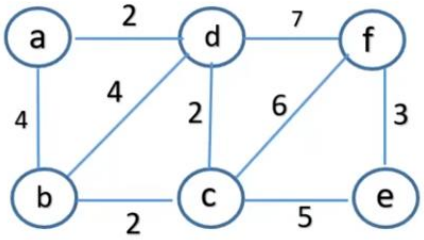


ad	2
bc	2
dc	2
fe	3
ab	4
bd	4
ce	5
cf	6
df	7

Create 6 disjoint sets :{a}, {b}, {c}, {d}, {e}, {f}

Final result :

V = {a, b, c, d, e, f }



ad	2 ✓
bc	2 ✓
dc	2 ✓
fe	3 ✓
→ ab	4 ✗
→ bd	4 ✗
→ ce	5 ✓
cf	6
df	7

Create 6 disjoint sets :{a}, {b}, {c}, {d}, {e}, {f}

{a, d}, {b}, {c}, {e}, {f}
{a, d}, {b, c}, {e}, {f}
{a, b, c, d}, {e}, {f}
{a, b, c, d}, {e, f}
{a, b, c, d, e, f}

Final result : ad | bc | dc | fe | ce

After processing, The final result will be: